

UNIT - IV

Operating System

Introduction

- Generally, a **Computer System** consists of the following components:
- **Computer Users** are the users who use the overall computer system.
- **Application Softwares** are the softwares which users use directly to perform different activities. These softwares are simple and easy to use like Browsers, Word, Excel, different Editors, Games etc. These are usually written in high-level languages, such as Python, Java and C++.
- **System Softwares** are the softwares which are more complex in nature and they are more near to computer hardware. These software are usually written in low-level languages like assembly language and includes **Operating Systems** (Microsoft Windows, macOS, and Linux), Compiler, and Assembler etc.
- **Computer Hardware** includes Monitor, Keyboard, CPU, Disks, Memory, etc.

Introduction of Operating System

- An Operating System (OS) is a collection of software that manages computer hardware resources and provides common services for computer programs.
- When you start using a Computer System then it's the Operating System (OS) which acts as an interface between you and the computer hardware.
- The operating system is really a low level **Software** which is categorized as a **System Software** and supports a computer's basic functions, such as memory management, tasks scheduling and controlling peripherals etc.

What System Software?

- System software provides an interface between the system hardware and the user.
- It lets the system understand the command that the user inputs.
- We can also say that the system software serves as an interface between the hardware in a system and application software.
- It is also called as general purpose software.
- The end user usually does not directly interact with the system software.
- The user only gets to interact with the GUI that has been created by the system software.

What is Application Software?

- We can define an application software as software written in a high-level language such as Java, C++, .net and VB for instance.
- This software is meant to meet some specific needs of the user. Some of the kinds of application software are designing software, a computing software, and an editing software.
- There is a specific purpose behind every application software.
- Application software runs on the platform created by the system software.
- We can say that application software provides a kind of intermediary between the end user and the software installed on a computer.
- Application software is not absolutely essential for running a system, but it makes the system more useful.
- The MS Office you find on Windows-based systems is a typical example of application software.

Difference between system and application software

System software is meant to manage the system resources. It serves as the platform to run application software.

System software is developed in a low-level language (assembly language for example)

System software automatically starts running once the system is turned on and stops when the system is shut down.

A system cannot even start without system software

System software is designed for a general purpose.

A typical example for a system software is Windows Operating System, Device driver, Translator etc

Application software helps perform a specific set of functions for which they have been designed.

Application software is developed in a high-level language such as Java, C++, .net and VB.

Application software runs as and when the user requests it.

Application software is user specific and it is not needed to run the system on the whole.

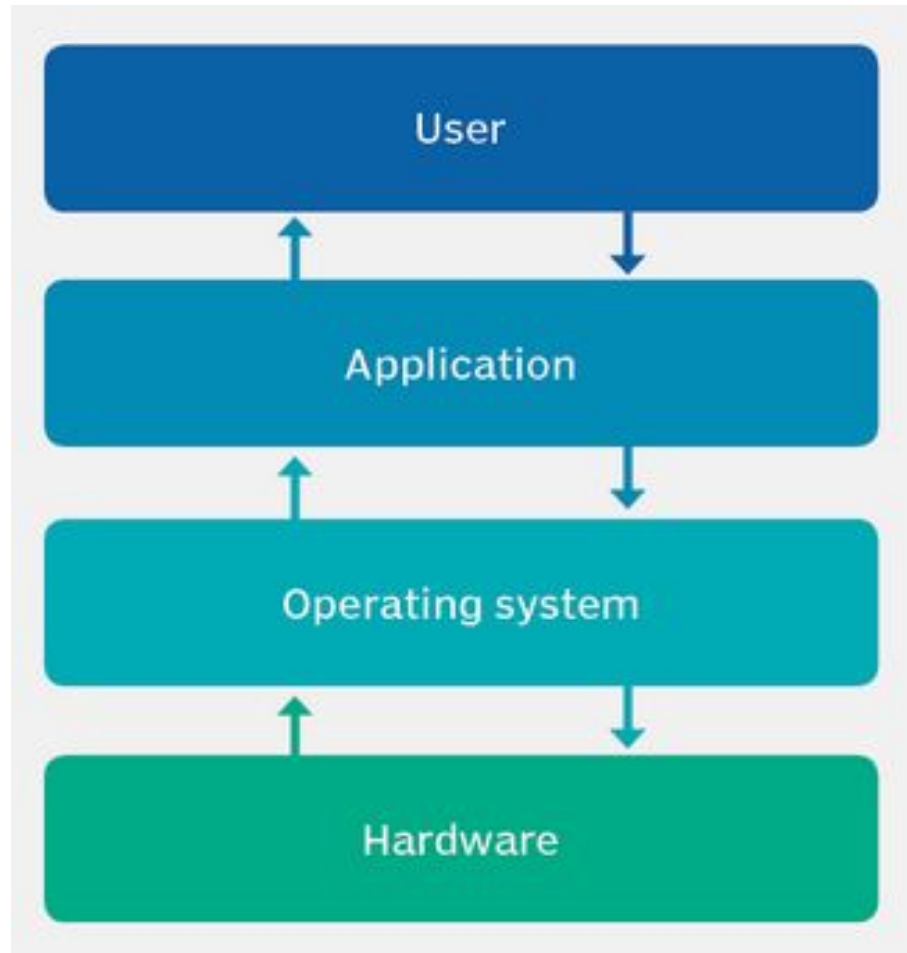
Application software carries a specific purpose.

Some characteristic examples for application software is MS Office, Photoshop and CorelDraw etc

What is Operating System

- An operating system is a program that acts as an interface between the software and the computer hardware.
- An operating system is a software which performs all the basic tasks like file management, memory management, process management, handling input and output, and controlling peripheral devices such as disk drives and printers.
- It is a specialized software that controls and monitors the execution of all other programs that reside in the computer, including application programs and other system software.
- Every computer system must have at least one operating system to run other programs.
- Applications like Browsers, MS Office, Notepad Games, etc., need some environment to run and perform its tasks.
- The OS helps you to communicate with the computer without knowing how to speak the computer's language.
- It is not possible for the user to use any computer or mobile device without having an operating system.

Placement of Operating System



History Of OS

- Operating systems were first developed in the late 1950s to manage tape storage
- The General Motors Research Lab implemented the first OS in the early 1950s for their IBM 701
- In the mid-1960s, operating systems started to use disks
- In the late 1960s, the first version of the Unix OS was developed
- The first OS built by Microsoft was DOS.
- It was built in 1981.
- The present-day popular OS Windows first came to existence in 1985 when a GUI was created and paired with MS-DOS.

Operating System - Examples

- There are plenty of Operating Systems available in the market which include paid and unpaid (Open Source). Following are the examples of the few most popular Operating Systems:
- **Windows:** This is one of the most popular and commercial operating systems developed and marketed by Microsoft. It has different versions in the market like Windows 8, Windows 10 etc and most of them are paid.
- **Linux** This is a Unix based and the most loved operating system first released on September 17, 1991 by Linus Torvalds. Today, it has 30+ variants available like Fedora, OpenSUSE, CentOS, UBuntu etc. Most of them are available free of charges though you can have their enterprise versions by paying a nominal license fee.
- **MacOS** This is again a kind of Unix operating system developed and marketed by Apple Inc. since 2001.
- **iOS** This is a mobile operating system created and developed by Apple Inc. exclusively for its mobile devices like iPhone and iPad etc.
- **Android** This is a mobile Operating System designed primarily for touchscreen mobile devices such as smartphones and tablets.

Objectives of Operating System

- **Efficiency**
- **Hardware abstraction**
- **Convenience**
- **System resource management**

Objectives of Operating System

Efficiency

- The operating system increases the production efficiency. This is because the system configuration takes less time. By default, the operating system handles system tasks such as allocating resources to processes and resolving conflicts between different programs and users. This saves the user time and results in a more efficient result.

Hardware abstraction

- The operating system performs a good job of hiding the computer's complex details. The user can fully utilise the computer hardware without having to cope with the accompanying difficulties. The operating system coordinates communication between user programs and computer hardware.

Objectives of Operating System

Convenience

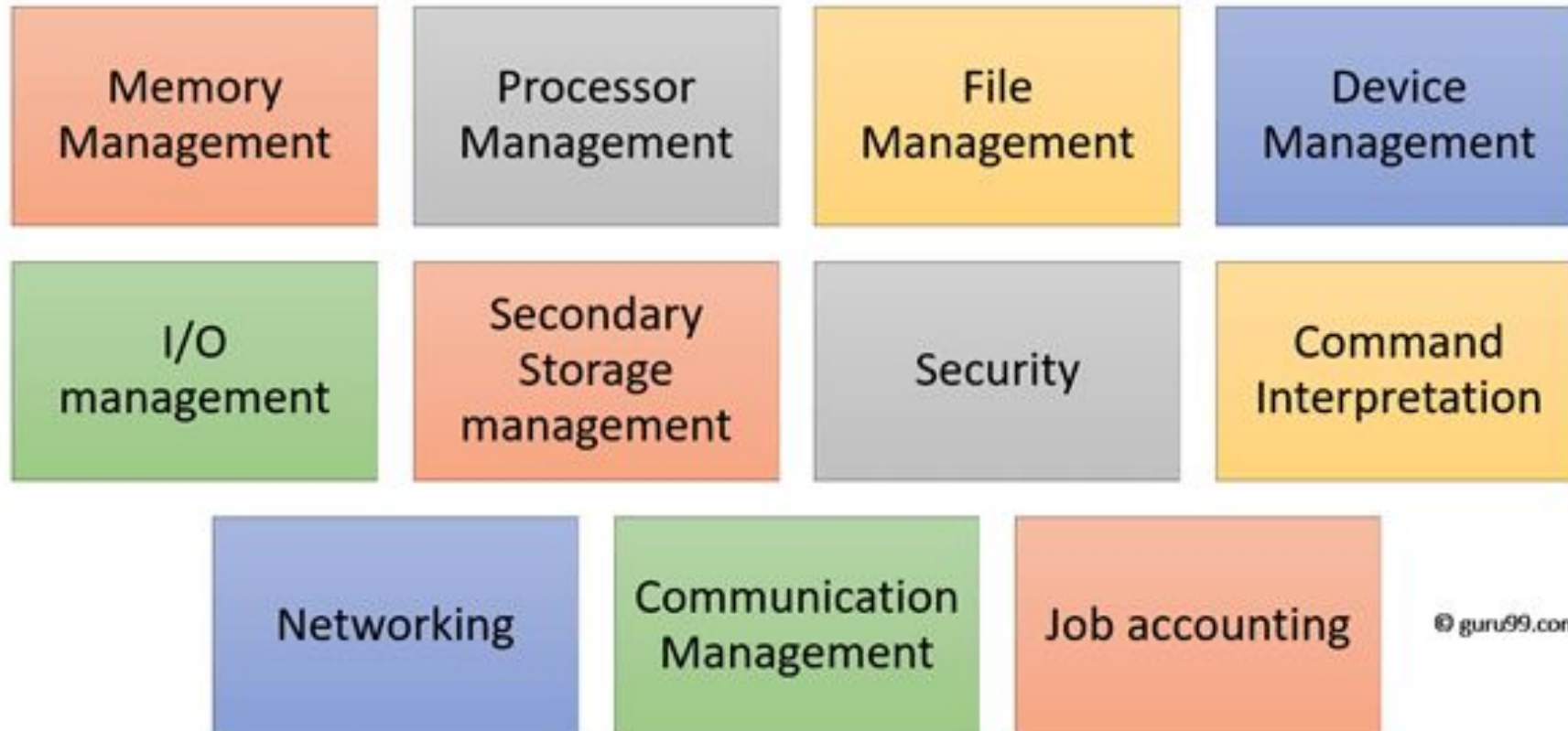
- In the absence of an operating system, users would have to deal with the hardware directly without access to the pre-configured utility packages that come with an operating system. This would make using a computer extremely inconvenient. Operating systems allow users to go right to work on the tasks they want to do without dealing with the burden of setting up the system beforehand.

System resource management

- The operating system serves as a neutral authority. It serves a management role in the computer system by ensuring equitable resource distribution among various operations and consumers.

Functions of Operating System

- Some typical operating system functions may include managing memory, files, processes, I/O system & devices, security, etc.
- Below are the main functions of Operating System:



Functions of Operating System

- **Process management:** Process management helps OS to create and delete processes. It also provides mechanisms for synchronization and communication among processes.
- **Memory management:** Memory management module performs the task of allocation and de-allocation of memory space to programs in need of this resources.
- **File management:** A file system is organized into directories for efficient or easy navigation and usage. These directories may contain other directories and other files. An Operating System carries out the following file management activities. It keeps track of where information is stored, user access settings and status of every file, and more... These facilities are collectively known as the file system.

Functions of Operating System

- **I/O System Management:** One of the main objects of any OS is to hide the peculiarities (a strange or unusual characteristic) of that hardware devices from the user and manage and grant the I/O devices whenever needed.
- **Secondary-Storage Management:** Systems have several levels of storage which includes primary storage, secondary storage, and cache storage. Instructions and data must be stored in primary storage or cache so that a running program can reference it.
- **Security:** The operating system uses password protection to protect user data and similar other techniques. it also prevents unauthorized access to programs and user data.
- **Command interpretation:** This module is interpreting commands given by the user and acting system resources to process that commands.

Functions of Operating System

- **Networking:** A distributed system is a group of processors which do not share memory, hardware devices, or a clock. The processors communicate with one another through the network.
- **Job accounting:** Keeping track of time & resource used by various job and users.
- **Communication management:** Coordination and assignment of compilers, interpreters, and another software resource of the various users of the computer systems.
- **Device Management:** Device management keeps tracks of all devices. This module also responsible for this task is known as the I/O controller. It also performs the task of allocation and de-allocation of the devices.

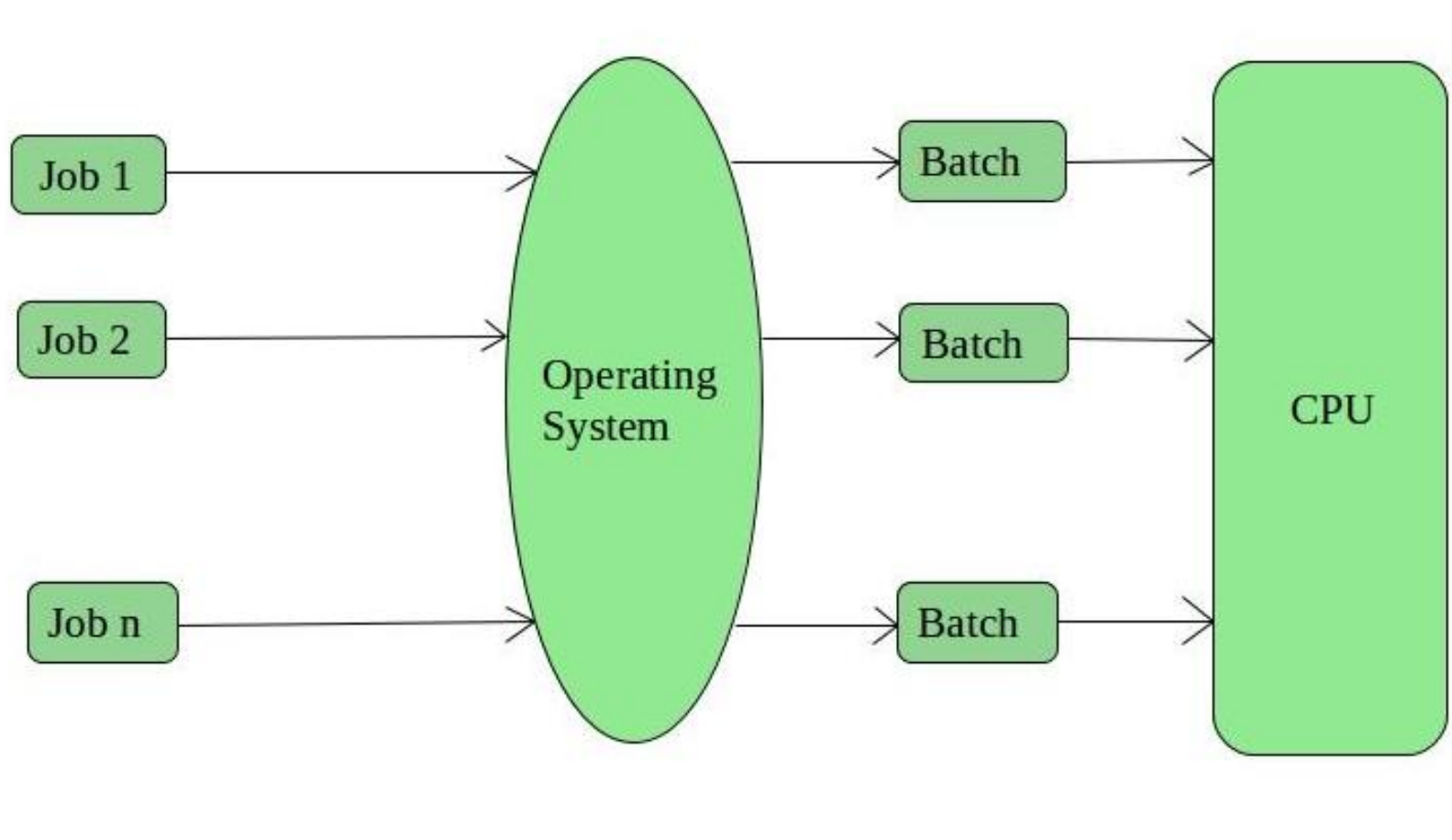
Types of Operating System

- 1) Batch Operating System
- 2) Time-Sharing Operating Systems
- 3) Distributed Operating System
- 4) Network Operating System
- 5) Real-Time Operating System

Batch Operating System

- The users of a batch operating system do not interact with the computer directly.
- Each user prepares his job on an off-line device like punch cards and submits it to the computer operator.
- To speed up processing, jobs with similar needs are batched together and run as a group.
- The programmers leave their programs with the operator and the operator then sorts the programs with similar requirements into batches.
- In this type, operating system does not interact with the computer directly.
- There is an operator which takes similar jobs having the same requirement and group them into batches. It is the responsibility of the operator to sort jobs with similar needs.

Batch Operating System



Problems - Batch Operating System

- Lack of interaction between the user and the job.
- CPU is often idle, because the speed of the mechanical I/O devices is slower than the CPU.
- Difficult to provide the desired priority.

Batch Operating System

Advantages

- It is very difficult to guess or know the time required for any job to complete. Processors of the batch systems know how long the job would be when it is in queue
- Multiple users can share the batch systems
- The idle time for the batch system is very less
- It is easy to manage large work repeatedly in batch systems

Batch Operating System

Disadvantages

- The computer operators should be well known with batch systems
- Batch systems are hard to debug
- It is sometimes costly
- The other jobs will have to wait for an unknown time if any job fails

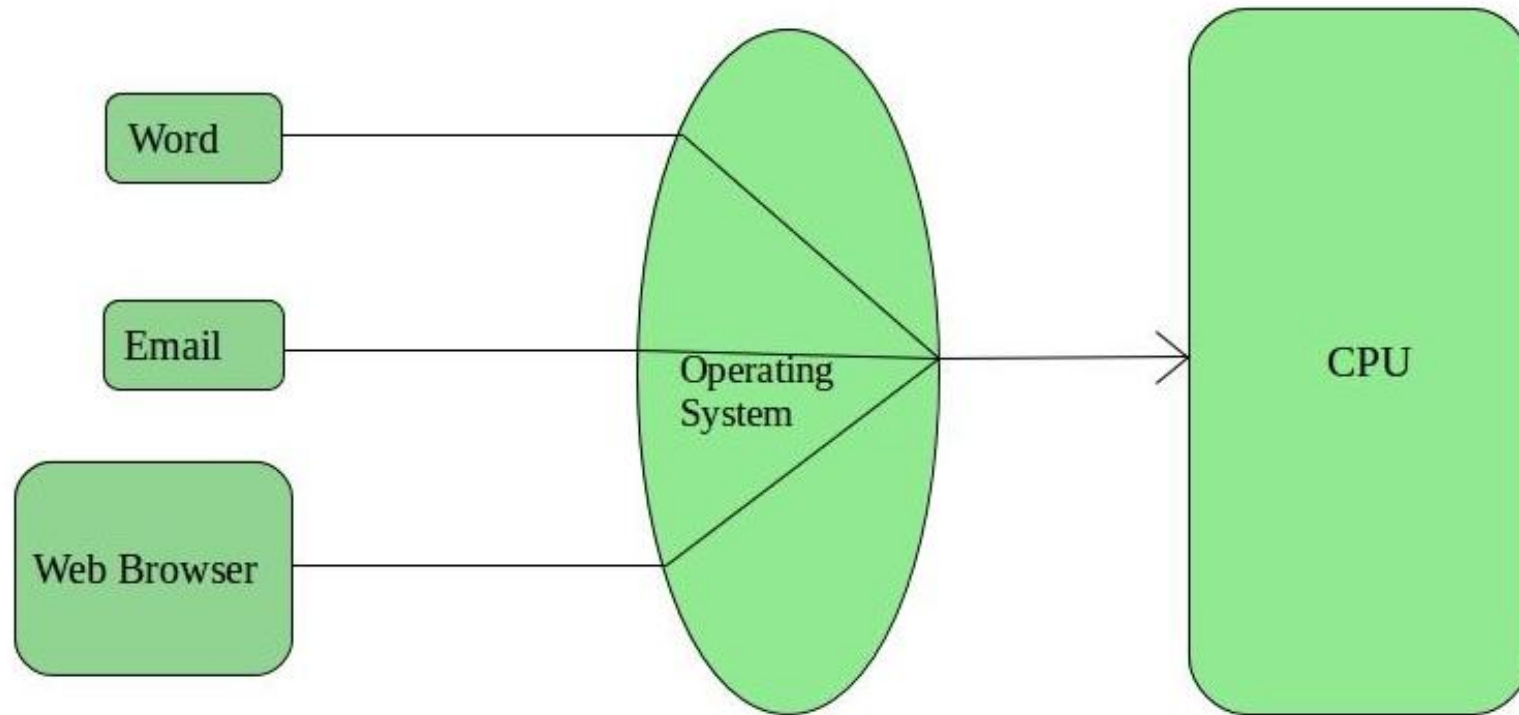
Time sharing Operating System

- Time-sharing is a technique which enables many people, located at various terminals, to use a particular computer system at the same time.
- Time-sharing or multitasking is a logical extension of multiprogramming.
- Processor's time which is shared among multiple users simultaneously (at the same time)
- The objective is to maximize processor use, whereas in Time-Sharing Systems, the objective is to minimize response time.

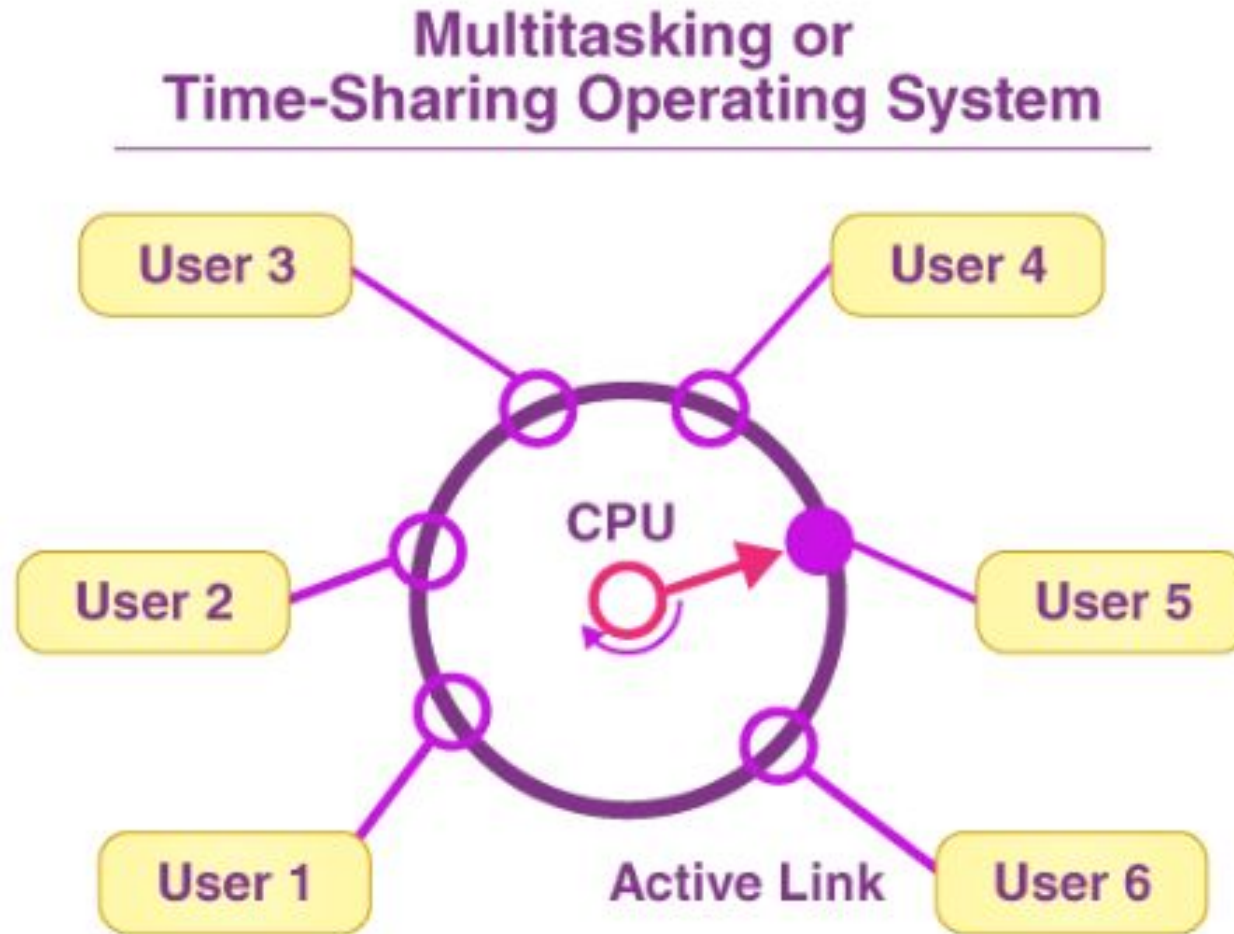
Time sharing Operating System

- Each task is given some time to execute so that all the tasks work smoothly.
- Each user gets the time of CPU as they use a single system.
- These systems are also known as Multitasking Systems.
- The task can be from a single user or different users also.
- The time that each task gets to execute is called quantum.
- After this time interval is over OS switches over to the next task.

Time sharing Operating System



Time sharing Operating System



Time sharing Operating System

- Multiprogramming is logically followed by time-sharing.
- Many processes are assigned to computer resources in time slots in this time-sharing operating system. The processor's time is split among numerous users in this scenario.
- The CPU changes between numerous jobs to complete them, but the shifts are frequent.
- As a result, the user can expect a quick answer. To give each user a little amount of time, the operating system utilizes CPU scheduling and multiprogramming.

Time sharing Operating System

Advantages

- Each task gets an equal opportunity
- Fewer chances of duplication of software
- CPU idle time can be reduced

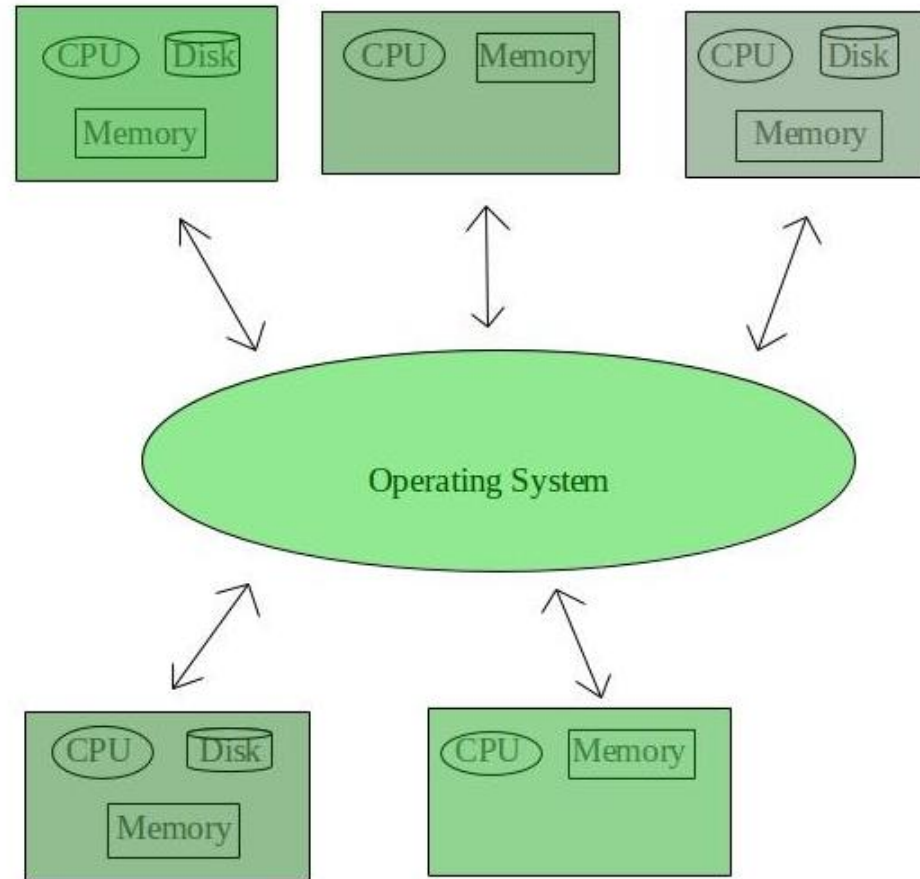
Disadvantages

- Reliability problem
- One must have to take care of the security and integrity of user programs and data
- Data communication problem
- **Examples:** Multics, Unix, etc.

Distributed Operating System

- Various autonomous interconnected computers communicate with each other using a shared communication network.
- Independent systems possess their own memory unit and CPU.
- These are referred to as **loosely coupled systems** or distributed systems.
- These system's processors differ in size and function.
- The major benefit of working with these types of the operating system is that it is always possible that one user can access the files or software which are not actually present on his system but some other system connected within this network i.e., remote access is enabled within the devices connected in that network.

Distributed Operating System



Distributed Operating System

Advantages:

- Failure of one will not affect the other network communication, as all systems are independent from each other
- Since resources are being shared, computation is highly fast and durable
- Load on host computer reduces.
- These systems are easily scalable as many systems can be easily added to the network
- Delay in data processing reduces

Distributed Operating System

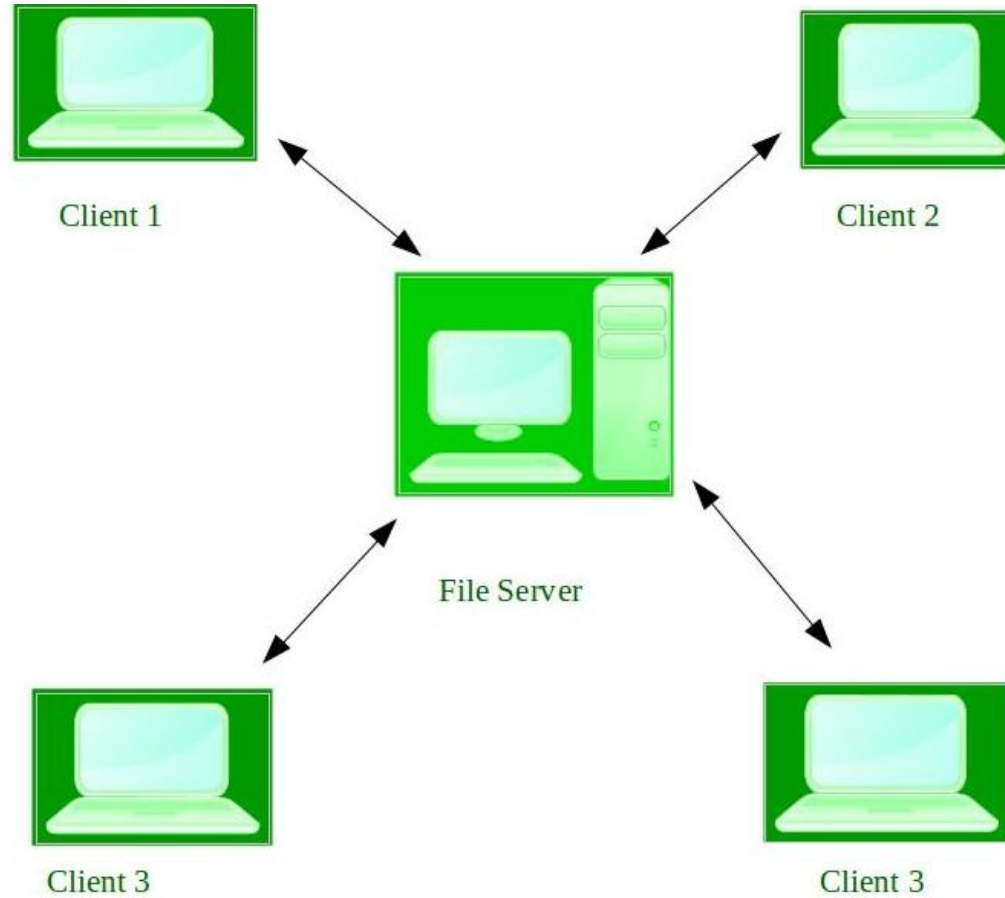
Disadvantages:

- Failure of the main network will stop the entire communication
- To establish distributed systems the language which is used are not well defined yet
- These types of systems are not readily available as they are very expensive. Not only that the underlying software is highly complex and not understood well yet
- **Examples of Distributed Operating System are- LOCUS**

Network Operating System

- These systems run on a server and provide the capability to manage data, users, groups, security, applications, and other networking functions.
- These types of operating systems allow shared access of files, printers, security, applications, and other networking functions over a small private network.
- One more important aspect of Network Operating Systems is that all the users are well aware of the underlying configuration, of all other users within the network, their individual connections, etc. and that's why these computers are popularly known as **tightly coupled systems**.

Network Operating System



Network Operating System

Advantages :

- Highly stable centralized servers
- Security concerns are handled through servers
- New technologies and hardware up-gradation are easily integrated into the system
- Server access is possible remotely from different locations and types of systems

Network Operating System

Disadvantages :

- Servers are costly
- User has to depend on a central location for most operations
- Maintenance and updates are required regularly
- **Examples :** Microsoft Windows Server 2003, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, and BSD, etc.

Real-Time Operating System

- These types of OSs serve real-time systems.
- A real-time system is defined as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment.
- The time taken by the system to respond to an input and display of required updated information is termed as the **response time**.
- So in this method, the response time is very less.
- **Real-time systems** are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Two types of Real-Time Operating System which are as follows:

- 1) Hard real-time systems**
- 2) Soft real-time systems**

Hard Real-Time Systems

- Hard real-time systems guarantee that critical tasks complete on time.
- These OSs are meant for applications where time constraints are very strict and even the shortest possible delay is not acceptable.
- In hard real-time systems, secondary storage is limited or missing and the data is stored in ROM.
- In these systems, virtual memory is almost never found.
- These systems are built for saving life like automatic parachutes or airbags which are required to be readily available in case of any accident.

Soft Real-Time Systems

Soft real-time systems are less restrictive.

A critical real-time task gets priority over other tasks and retains the priority until it completes.

Soft real-time systems have limited utility than hard real-time systems.

For example, multimedia, virtual reality, Advanced Scientific Projects like undersea exploration and planetary rovers, etc.

Real-Time Operating System - Advantages

- **Maximum Consumption**: Maximum utilization of devices and system, thus more output from all the resources
- **Task Shifting**: The time assigned for shifting tasks in these systems are very less. For example, in older systems, it takes about 10 microseconds in shifting one task to another, and in the latest systems, it takes 3 microseconds.
- **Focus on Application**: Focus on running applications and less importance to applications which are in the queue.
- **Real-time operating system in the embedded system**: Since the size of programs are small, RTOS can also be used in embedded systems like in transport, GPS systems, ATMs and others.
- **Error Free**: These types of systems are error-free.
- **Memory Allocation**: Memory allocation is best managed in these types of systems.

Real-Time Operating System- DisAdvantages

- **Limited Tasks**: Very few tasks run at the same time and their concentration is very less on few applications to avoid errors.
- **Use heavy system resources**: Sometimes the system resources are not so good and they are expensive as well.
- **Complex Algorithms**: The algorithms are very complex and difficult for the designer to write on.
- **Device driver and interrupt signals**: It needs specific device drivers and interrupts signals to respond earliest to interrupts.
- **Examples of Real-Time Operating Systems are**: Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

PROCESS DESCRIPTION AND CONTROL

Process

- Bill is an Engineering student in his first year of college.
- With a lot of assignments piling up and deadlines floating in, he realizes that he needs to do something to manage his time better.
- So, every night, Bill notes down the tasks that he needs to do the next day in the order of execution.
- Here, Bill writes a program that he would execute when the time comes. At the beginning of every day, he refers to his schedule and follows the steps in the order mentioned.
- This is **called a process**.
- When you write a set of actions in sequential order, it is called a program. The **program, when in execution is called a process**.

Process

- A process is basically a program in execution.
- When a program is loaded into the memory and it becomes a process.
- The process executes when it changes the state. The state of a process is defined by the current activity of the process.
- For example, when we write a program in C or C++ and compile it, the compiler creates binary code. The original code and binary code are both programs. When we actually run the binary code, it becomes a process.

PROCESS - STATES

- **New** – A program which is going to be picked up by the OS into the main memory is called a new process.
- **Ready** - Whenever a process is created, it directly enters in the ready state, in which, it waits for the CPU to be assigned. The OS picks the new processes from the secondary memory and put all of them in the main memory. The processes which are ready for the execution and reside in the main memory are called ready state processes. There can be many processes present in the ready state.
- **Running** – One of the processes from the ready state will be chosen by the OS depending upon the scheduling algorithm. Hence, if we have only one CPU in our system, the number of running processes for a particular time will always be one. If we have n processors in the system then we can have n processes running simultaneously.

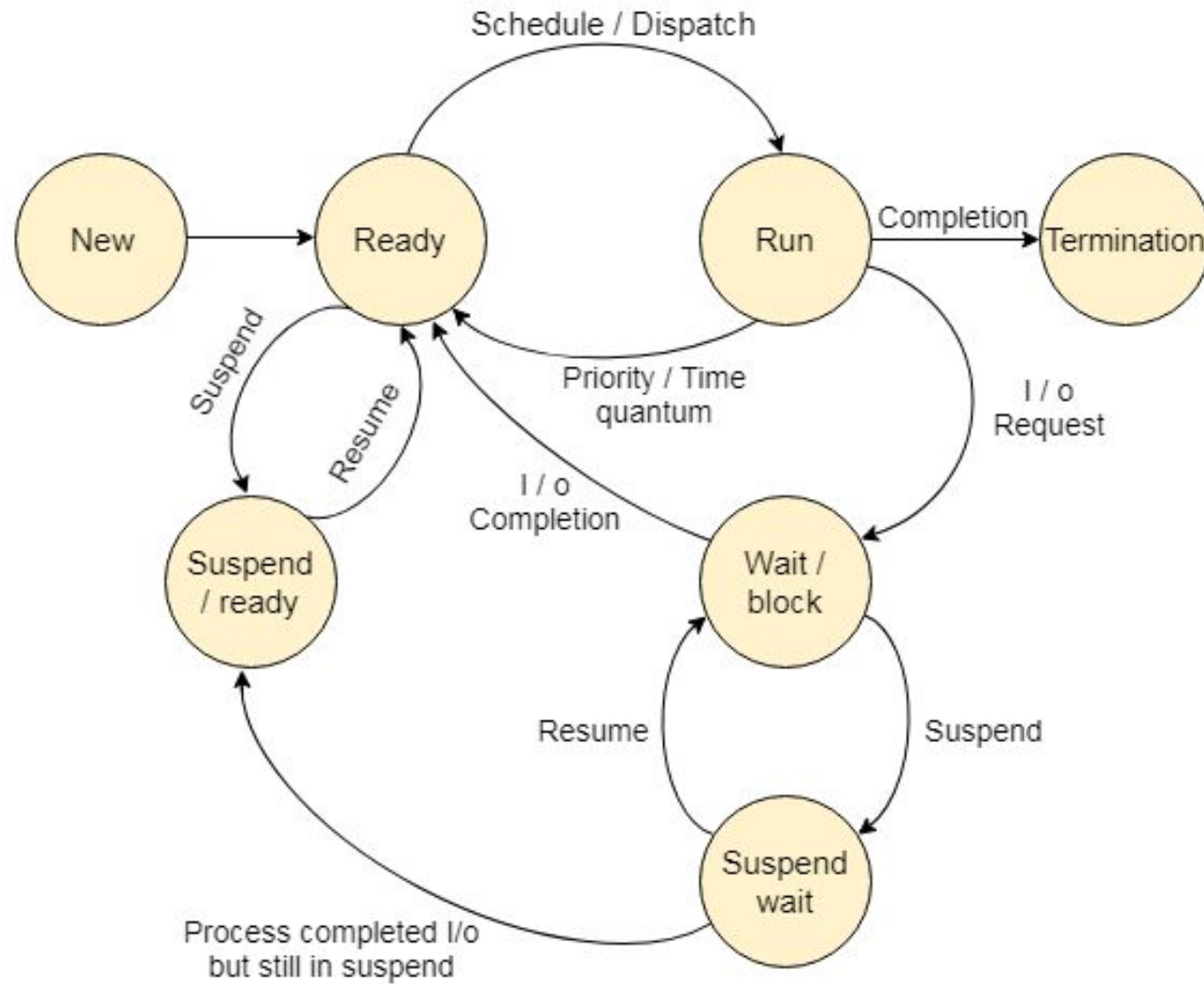
PROCESS - STATES

- **Block or Waiting** – From the Running state, a process can make the transition to the block or wait state depending upon the scheduling algorithm or the intrinsic behavior of the process.
- When a process waits for a certain resource to be assigned or for the input from the user then the OS move this process to the block or wait state and assigns the CPU to the other processes.
- **Completion or termination** - When a process finishes its execution, it comes in the termination state. All the context of the process (Process Control Block) will also be deleted the process will be terminated by the Operating system.

PROCESS - STATES

- **Suspend ready** - A process in the ready state, which is moved to secondary memory from the main memory due to lack of the resources (mainly primary memory) is called in the suspend ready state.
- **Suspend wait** - Instead of removing the process from the ready queue, it's better to remove the blocked process which is waiting for some resources in the main memory. Since it is already waiting for some resource to get available hence it is better if it waits in the secondary memory and make room for the higher priority process. These processes complete their execution once the main memory gets available and their wait is finished.

Process States



Operations on the Process

1. Creation

Once the process is created, it will be ready and come into the ready queue (main memory) and will be ready for the execution.

2. Scheduling

Out of the many processes present in the ready queue, the Operating system chooses one process and start executing it. Selecting the process which is to be executed next, is known as scheduling.

3. Execution

Once the process is scheduled for the execution, the processor starts executing it. Process may come to the blocked or wait state during the execution then in that case the processor starts executing the other processes.

4. Deletion/killing

Once the purpose of the process gets over then the OS will kill the process. The Context of the process (PCB) will be deleted and the process gets terminated by the Operating system.

Process control block

- A Process Control Block is a data structure maintained by the Operating System for every process.
- To identify the processes, the OS assigns a process identification number (PID) to each process.
- As the operating system supports multi-programming, it needs to keep track of all the processes.
- For this task, the process control block (PCB) is used to track the process's execution status
- Each block of memory contains information about the process state, program counter, stack pointer, status of opened files, scheduling algorithms, etc.
- When the process makes a transition from one state to another, the operating system must update information in the process's PCB.

Process control block (PCB)

Pointer
Process State
Process Number
Program Counter
Registers
Memory Limits
Open File Lists
Misc. Accounting and Status Data

Process control block

- **Pointer** – It is a stack pointer which is required to be saved when the process is switched from one state to another to retain the current position of the process.
- **Process state** – It stores the respective state of the process.
- **Process number** – Every process is assigned with a unique id known as process ID or PID which stores the process identifier.
- **Program counter** – It stores the counter which contains the address of the next instruction that is to be executed for the process.
- **Register** – These are the CPU registers which includes: accumulator, base, registers and general purpose registers.
- **Memory limits** – This field contains the information about memory management system used by operating system. This may include the page tables, segment tables etc
- **Open files list** – This information includes the list of files opened for a process.
- **Accounting information** - This includes the amount of CPU used for process execution, time limits, execution ID etc.

Modes of Execution

- The dual-mode operations in the operating system protect the operating system from illegal users.
- We accomplish this defense by designating some of the system instructions as privileged instructions that can cause harm.
- The hardware only allows for the execution of privileged instructions in kernel mode.
- An example of a privileged instruction is the command to switch to user mode.
- Other examples include monitoring of I/O, controlling timers and handling interruptions.
- To ensure proper operating system execution, we must differentiate between machine code execution and user-defined code.
- Most computer systems have embraced offering hardware support that helps distinguish between different execution modes.
- We have two modes of the operating system: ***user mode*** and ***kernel mode***.
- Mode bit is required to identify in which particular mode the current instruction is executing. If the mode bit is 1, it operates user mode, and if the mode bit is 0, it operates in kernel mode.
- **NOTE: At the booting time of the system, it always starts with the kernel mode**

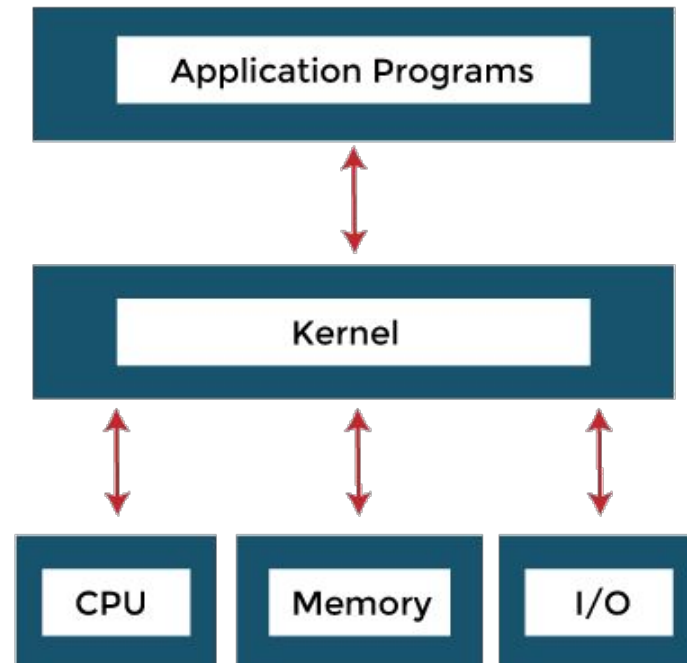
Modes of Execution

- Types of Dual Mode in Operating System
- The operating system has two modes of operation to ensure it works correctly: **user mode and kernel mode.**
- **1. User Mode**
 - When the computer system runs user applications like file creation or any other application program in the User Mode, this mode does not have direct access to the computer's hardware.
 - For performing hardware related tasks, like when the user application requests for a service from the operating system or some interrupt occurs, in these cases, the system must switch to the Kernel Mode.
 - The mode bit of the user mode is 1. This means that if the mode bit of the system's processor is 1, then the system will be in the User Mode.

Modes of Execution

- **2. Kernel Mode**

- All the bottom level tasks of the Operating system are performed in the Kernel Mode.
- As the Kernel space has direct access to the hardware of the system, so the kernel-mode handles all the processes which require hardware support.
- Apart from this, the main functionality of the Kernel Mode is to execute privileged instructions.

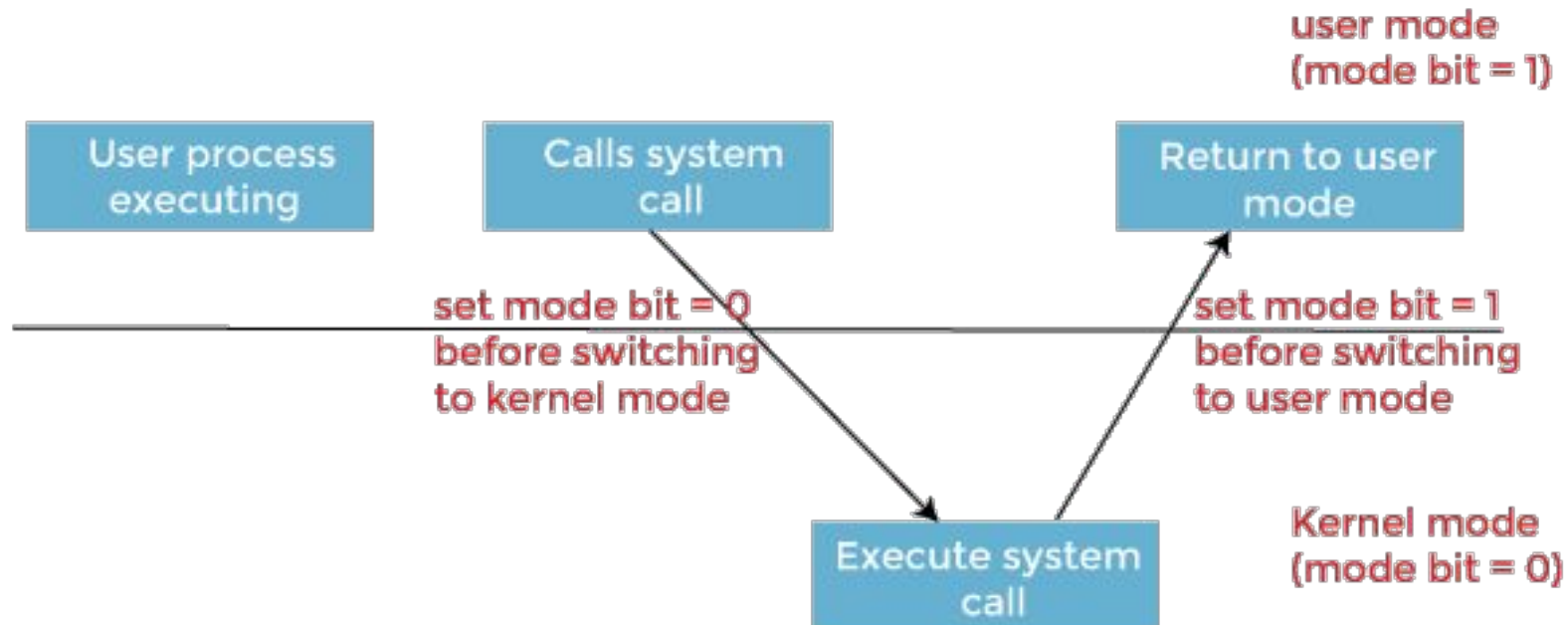


Modes of Execution

- These privileged instructions are not provided with user access, and that's why these instructions cannot be processed in the User mode.
- So, all the processes and instructions that the user is restricted to interfere with are executed in the Kernel Mode of the Operating System.
- The mode bit for the Kernel Mode is 0.
- So, for the system to function in the Kernel Mode, the Mode bit of the processor must be equal to 0.

Modes of Execution

- Example
- With the mode bit, we can distinguish between a task executed on behalf of the operating system and one executed on behalf of the user.



Modes of Execution

- When the computer system executes on behalf of a user application, the system is in ***user mode***.
- However, when a user application requests a service from the operating system via a system call, it must transition from ***user*** to ***kernel mode*** to fulfill the request. As we can say, this architectural enhancement is useful for many other aspects of system operation.
- At system boot time, the hardware starts in ***kernel mode***.
- The operating system is then loaded and starts user applications in ***user mode***.
- Whenever a trap or interrupt occurs, the hardware switches from ***user mode*** to ***kernel mode***, changing the mode bit's state to 0.
- Thus, whenever the operating system gains control of the computer, it is in ***kernel mode***.
- The system always ***switches to user mode*** by setting the mode bit to 1 before passing control to a user program.

Modes of Execution

- Need for Dual-Mode Operations
- Certain types of processes are to be made hidden from the user, and certain tasks that do not require any type of hardware support. Using the ***dual mode*** of the OS, these tasks can be deal with separately.
- Also, the Operating System needs to function in the ***dual mode*** because the Kernel Level programs perform all the bottom level functions of the OS like process management, Memory management, etc.
- If the user alters these, then this can cause an entire system failure. So, for specifying the access to the users only to the tasks of their use, Dual Mode is necessary for an Operating system.
- So, whenever the system works on the user applications, it is in the User mode.
- Whenever the user requests some hardware services, a transition from User mode to Kernel mode occurs, and this is done by changing the mode bit from 1 to 0.
- And for returning back into the User mode, the mode bit is again changed to 1.

INTRODUCTION TO SCHEDULING

Scheduling

- The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy.
- Process scheduling is an essential part of a Multiprogramming operating systems.
- Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.
- The operating system schedules the process so that the CPU always has one process to execute. This reduces the CPU's idle time and increases its utilization.
- The part of OS that allots the computer resources to the processes is termed as a **scheduler**. It uses **scheduling algorithms** to decide which process it must allot to the CPU.

Need of Scheduling

- Like in the morning you wake up, have breakfast, work for some hours take lunch, again resume your work, at night have dinner and go to bed. Doing this way, you make your day more logical and efficient.
- Similarly, the operating system schedules the processes to achieve maximum CPU utilization. The processor can keep several processes in the main memory in the multiprogramming environment. And some or the other processes must be running at all times.
- As we know a typical process has both I/O times as well as CPU time.
- When a process completes its CPU time and now needs to perform I/O operation. The operating system takes the CPU from this process.
- And provides it to the other process waiting for the CPU.
- In this way, the operating system manages the CPU resources in an efficient way.

Need of Scheduling

- If most of the running processes change their state from running to waiting then there may always be a possibility of deadlock in the system.
- A deadlock happens in operating system when two or more processes need some resource to complete their execution that is held by the other process.
- Hence to reduce this overhead, the OS needs to schedule the jobs to get the optimal utilization of CPU and to avoid the possibility to deadlock.

Categories of Scheduling

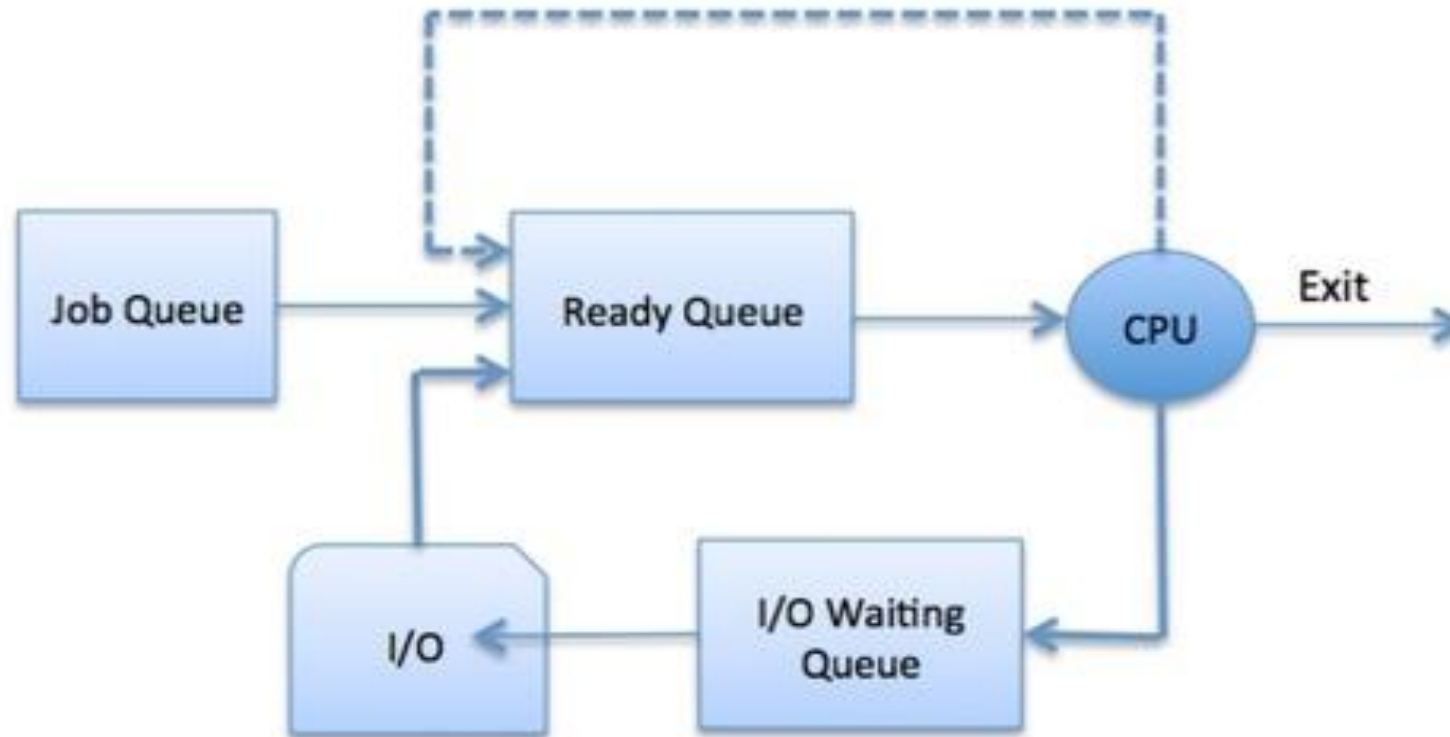
- There are two categories of scheduling:
- **Non-preemptive:** Here the resource can't be taken from a process until the process completes execution.
- **Preemptive:** Here the OS allocates the resources to a process for a fixed amount of time. During resource allocation, the process switches from running state to ready state or from waiting state to ready state. This switching occurs as the CPU may give priority to other processes and replace the process with higher priority with the running process.

Process Scheduling Queues

The Operating System maintains the following important process scheduling queues –

- **Job queue** – It contains all of the system's processes.
- **Ready queue** – This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.
- **Device queues** – The processes which are blocked due to unavailability of an I/O device constitute this queue.

Process Scheduling Queues



Schedulers

- Schedulers are special system software which handle process scheduling in various ways.
- Their main task is to select the jobs to be submitted into the system and to decide which process to run.

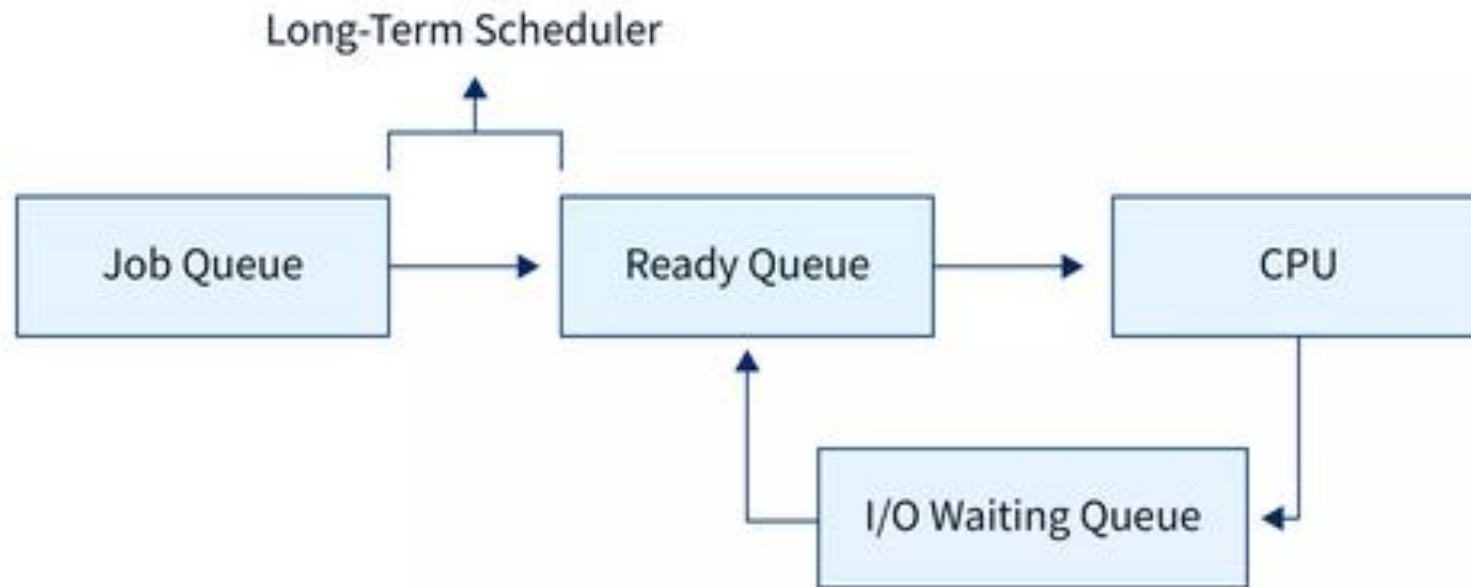
Schedulers are of three types –

1. Long-Term Scheduler
2. Short-Term Scheduler
3. Medium-Term Scheduler

Long Term Scheduler

- A long-term scheduler is a scheduler that is responsible for bringing processes from the **JOB** queue (**or secondary memory**) into the **READY** queue (**or main memory**).
- A long-term scheduler determines which programs will enter into the **RAM for processing by the CPU**.
- Long-term schedulers are also called **Job Schedulers**.
- They are responsible for the degree of multiprogramming, **i.e., managing the total processes present in the READY queue**.
- Some operating systems do not have a long term scheduler. For example, time-sharing operating systems **like Windows and UNIX usually don't have a long term scheduler**. These systems put all the processes in the main memory for the **short term scheduler**.

Long Term Scheduler



Short Term Scheduler

- It is also called as **CPU scheduler**.
- CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.
- It is the change of ready state to running state of the process.
- Short-term schedulers, also known as dispatchers, make the decision of which process to execute next.
- Short-term schedulers are faster than long-term schedulers.

Medium Term Scheduler

- It removes the processes from the memory.
- The medium-term scheduler is in-charge of handling the swapped out-processes.
- A running process may become suspended if it makes an I/O request.
- A suspended processes cannot make any progress towards completion.
- In this condition, to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage.
- This process is called **swapping**, and the process is said to be swapped out or rolled out.
- Swapping may be necessary to improve the process mix.

I/O Term Scheduler

- **I/O Scheduler** functions similar to Process scheduler, it allocates the devices, control units, and communication channels.
- However, under heavy load of I/O requests, Scheduler must decide what request should be served first and for that we multiple queues to be managed by OS.
- The major difference between **Process scheduler** and **I/O scheduler** is that I/O requests are not prevented.
- Once the channel program has started, it's allowed to continue to completion.
- The job of I/O scheduler is to make the disk reads and writes appropriately so as to take minimum time/seeking. **It manages the I/O to the disks.**
- The disk is considered to be very slow as compared to main memory, so if the reads/writes are honored in the way they arrive, it could possibly require a lot of seeking to random places, which would be very expensive and hit the performance badly.
- The I/O scheduler clubs read/write requests at more or less neighboring tracks before it seeks.

Scheduling Algorithms

A Process Scheduler schedules different processes to be assigned to the CPU based on particular scheduling algorithms.

1. First-Come, First-Served (FCFS) Scheduling
2. Shortest-Job-Next (SJN) Scheduling
3. Round Robin(RR) Scheduling

First-Come, First-Served (FCFS) Scheduling

- Jobs are executed according to their arrival time.
- Its implementation is based on FIFO queue.
- In this, the process that comes first will be executed first and next process starts only after the previous gets fully executed.
- FCFS scheduling may cause the problem of starvation if the burst time of the first process is the longest among all the jobs.
- **Note: The total amount of time required by the CPU to execute the whole process is called the Burst Time.**

First-Come, First-Served (FCFS) Scheduling

- **Advantages of FCFS**

- Simple
- Easy
- First come, First serve

- **Disadvantages of FCFS**

- The scheduling method is non preemptive, the process will run to the completion.
- Due to the non-preemptive nature of the algorithm, the problem of starvation may occur.
- Although it is easy to implement, but it is poor in performance since the average waiting time is higher as compare to other scheduling algorithms.

First-Come, First-Served (FCFS) Scheduling Example

The Turnaround time and the waiting time are calculated by using the following formula.

- **Turn Around Time** = Completion Time - Arrival Time
- It is the time interval from the time of submission of a process to the time of the completion of the process.
- The difference b/w Completion Time and Arrival Time is called Turnaround Time.
- **Waiting Time** = Turnaround time - Burst Time
- The time spent by a process waiting in the ready queue for getting the CPU.
- The time difference b/w Turnaround Time and Burst Time is called Waiting Time.

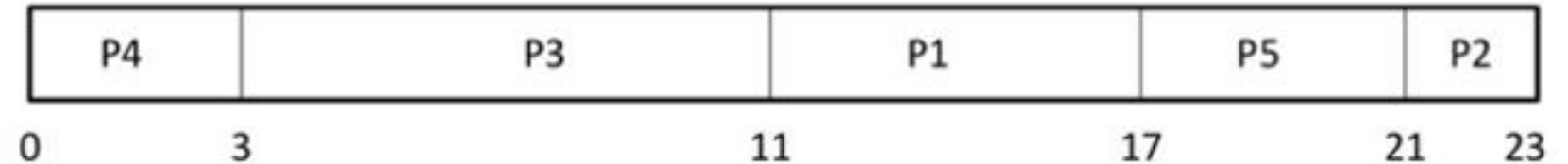
First-Come, First-Served (FCFS) Example

Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

First-Come, First-Served (FCFS) Example

Waiting time = Start time – arrival time

- $P4 = 0 - 0 = 0$
- $P3 = 3 - 1 = 2$
- $P1 = 11 - 2 = 9$
- $P5 = 17 - 4 = 13$
- $P2 = 21 - 5 = 16$



Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

Average Waiting Time = $(0 + 2 + 9 + 13 + 16) / 5$

FCFS - Example1

- Generate a time line(Gantt chart) and calculate the average time.
- **Waiting time = Start time – arrival time**

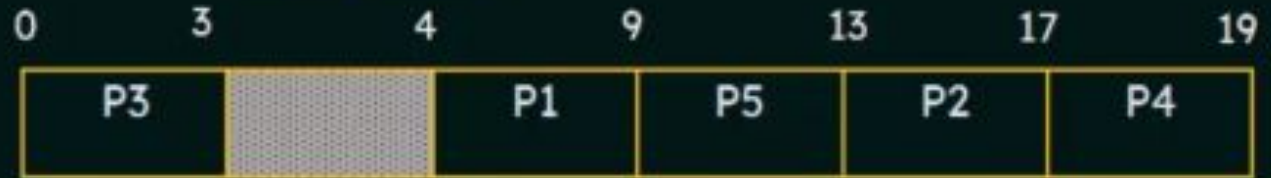
Process Id	Arrival time	Burst time
P1	3	4
P2	5	3
P3	0	2
P4	5	1
P5	4	3

FCFS - Example1

Solution:

Process ID	Arrival Time	Burst Time
P1	4	5
P2	6	4
P3	0	3
P4	6	2
P5	5	4

Gantt Chart:



The shaded box represents the idle time of CPU

Turn Around time = Completion time - Arrival time

Waiting time = Turn Around time - Burst time

Process ID	Completion Time	Turnaround Time	Waiting Time
P1	9	$9 - 4 = 5$	$5 - 5 = 0$
P2	17	$17 - 6 = 11$	$11 - 4 = 7$
P3	3	$3 - 0 = 3$	$3 - 3 = 0$

FCFS - Example1

Process ID	Completion Time	Turnaround Time	Waiting Time
P1	9	$9 - 4 = 5$	$5 - 5 = 0$
P2	17	$17 - 6 = 11$	$11 - 4 = 7$
P3	3	$3 - 0 = 3$	$3 - 3 = 0$
P4	19	$19 - 6 = 13$	$13 - 2 = 11$
P5	13	$13 - 5 = 8$	$8 - 4 = 4$

Now,

Average Turn Around time $= (5 + 11 + 3 + 13 + 8) / 5$
 $= 40 / 5$
 $= 8 \text{ units}$

Average waiting time $= (0 + 7 + 0 + 11 + 4) / 5$
 $= 22 / 5$
 $= 4.4 \text{ units}$

FCFS - Example2

Generate a time line and calculate the average time.

Waiting time = Start time – arrival time

Process	Burst Time	Arrival time
P1	6	2
P2	8	1
P3	3	0
P4	4	4

FCFS - Example3

Generate a time line and calculate the average time.

Waiting time = Start time – arrival time

Process	Arrival Time	Execute Time
P0	0	5
P1	1	3
P2	2	8
P3	3	6

FCFS - Example4

Generate a time line and calculate the average time.
Here arrival time of all process is 0

PROCESS	BURST TIME
P1	21
P2	3
P3	6
P4	2

FCFS - Example5

Generate a time line and calculate the average time.

Processes	Arrival Time	Burst Time
P1	0	4
P2	1	3
P3	2	1
P4	3	2
P5	4	5

Shortest Job Next (SJN)

- SJF scheduling algorithm, schedules the processes according to their burst time
- In SJF scheduling, the process with the lowest burst time, among the list of available processes in the ready queue, is going to be scheduled next.
- Best approach to minimize waiting time.
- It may cause starvation if shorter processes keep coming.

Shortest Job Next (SJN)

- **Advantages of SJF:**
- SJF is better than the First come first serve(FCFS) algorithm as it reduces the average waiting time.
- SJF is generally used for long term scheduling
- It is suitable for the jobs running in batches, where run times are already known.

Shortest Job Next (SJN)

- **Disadvantages of SJF:**
- May suffer with the problem of starvation
- It is not implementable because the exact Burst time for a process can't be known in advance.

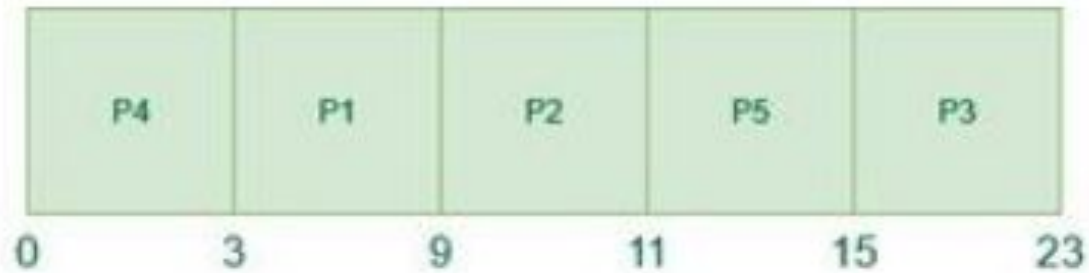
Shortest Job Next (SJN) - Example

Process	Burst Time	Arrival Time
P1	6 ms	2 ms
P2	2 ms	5 ms
P3	8 ms	1 ms
P4	3 ms	0 ms
P5	4 ms	4 ms

- **Waiting time** = Start time – Arrival Time

Shortest Job Next (SJN) - Example

Process	Burst Time	Arrival Time
P1	6 ms	2 ms
P2	2 ms	5 ms
P3	8 ms	1 ms
P4	3 ms	0 ms
P5	4 ms	4 ms



- **Waiting time** = Start time – Arrival Time
- $P4 = 0 - 0 = 0$
- $P1 = 3 - 2 = 1$
- $P2 = 9 - 5 = 4$
- $P5 = 11 - 4 = 7$
- $P3 = 15 - 1 = 14$
- **Average Waiting Time** = $0 + 1 + 4 + 7 + 14 / 5 = 26 / 5 = 5.2$

SJN / FCFS – Example 1

Generate a time line and calculate the average time.

Waiting time = Start time– Arrival Time

Process Queue	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

SJN / FCFS - Example2

Generate a time line and calculate the average time.

Waiting time = Start time – Arrival Time

Process	Arrival Time	Burst Time
P1	0	3
P2	0	6
P3	2	3
P4	3	2

SJN / FCFS -Example3

Generate a time line and calculate the average time.

Waiting time = Start time– Arrival Time

Process ID	Burst Time	Arrival Time
P0	8	5
P1	5	0
P2	9	4
P3	2	1

SJN / FCFS -Example 4

Generate a time line and calculate the average time.

Waiting time = Start time– Arrival Time

Process	Arrival Time	Execute Time
P0	0	5
P1	1	3
P2	2	8
P3	3	6

SJN / FCFS -Example 5

Generate a time line and calculate the average time.

Waiting time = Start time– Arrival Time

PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

SJN / FCFS –Example 6

Generate a time line and calculate the average time.

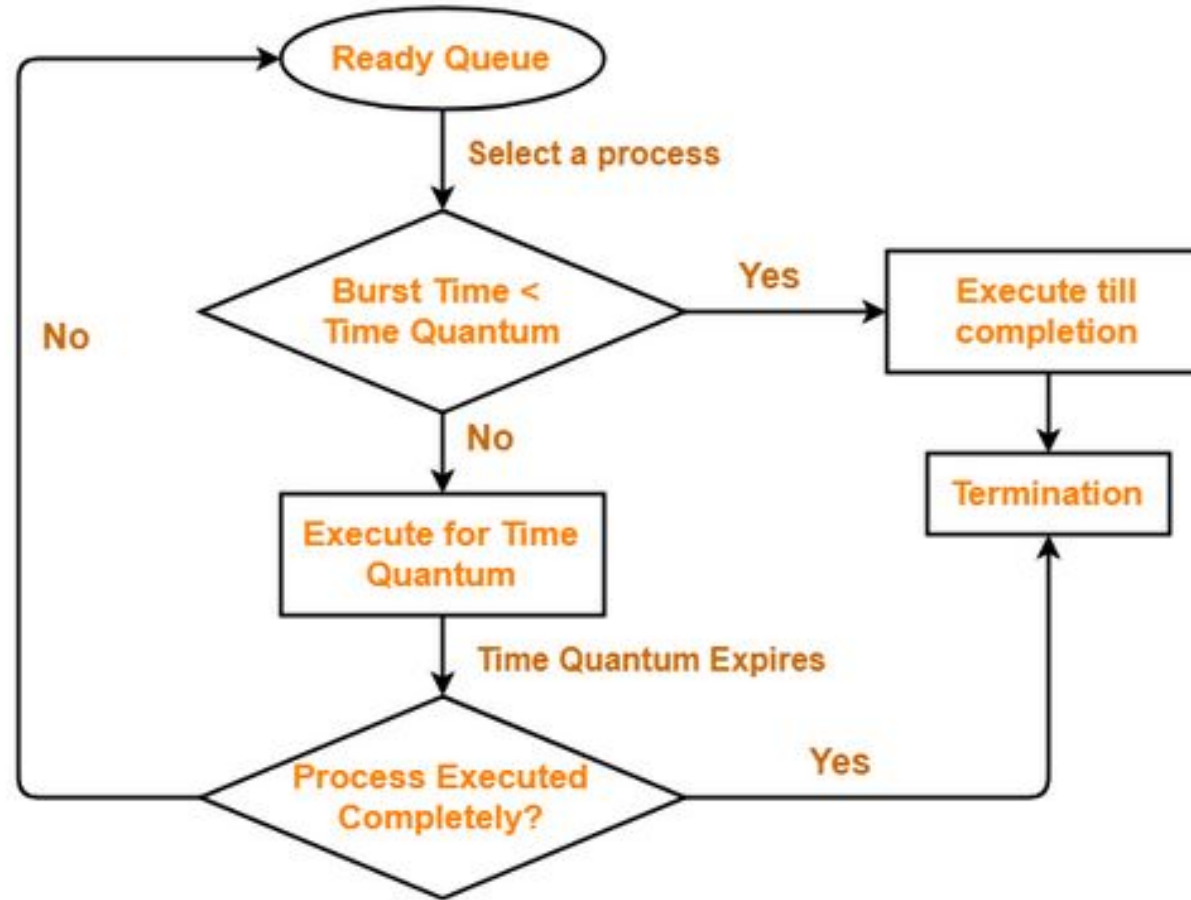
Waiting time = Start time– Arrival Time

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Round Robin Scheduling

- Round Robin CPU Algorithm focuses on Time Sharing technique.
- Every process gets executed in a **cyclic way**.
- Each process is provided a fix time to execute, it is called a **time quantum / time slice**.
- Each process present in the ready queue is assigned the CPU for that time quantum, if the execution of the process is completed during that time then the process will **terminate** else the process will go back to the **ready queue** and waits for the next turn to complete the execution.
- The higher the time quantum, the higher the response time in the system.
- The lower the time quantum, the higher the context switching overhead in the system.
- All the jobs get a fare allocation of CPU.

Round Robin Scheduling



Round Robin Scheduling - Example

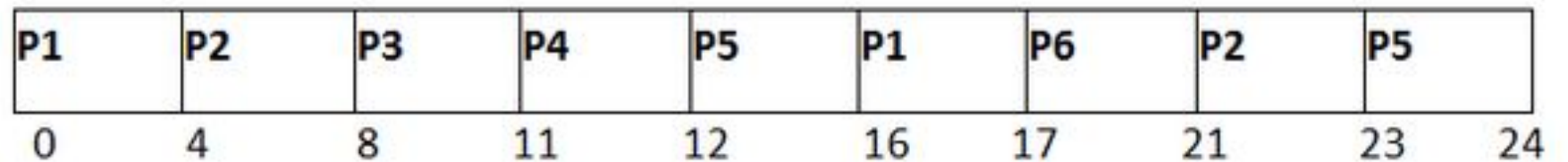
Quantum = 4

Process ID	Arrival Time	Burst Time
1	0	5
2	1	6
3	2	3
4	3	1
5	4	5
6	6	4

Round Robin Scheduling - Example

Process ID	Arrival Time	Burst Time
1	0	5
2	1	6
3	2	3
4	3	1
5	4	5
6	6	4

Gantt chart/time line chart:



Round Robin Scheduling - Example

- Turn Around Time = Completion Time - Arrival Time
- Waiting Time = Turn Around Time - Burst Time

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	0	5	17	17	12
2	1	6	23	22	16
3	2	3	11	9	6
4	3	1	12	9	8
5	4	5	24	20	15
6	6	4	21	15	11

Round Robin Scheduling – Example1

Quantum = 3

Process Id	Arrival time	Burst time
P1	0	4
P2	1	5
P3	2	2
P4	3	1
P5	4	6
P6	6	3

Round Robin Scheduling – Example2

Quantum = 2

Process Id	Arrival time	Burst time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

Round Robin Scheduling – Example3

Quantum = 4

Process ID	Arrival Time	Burst Time
P1	0	5
P2	1	6
P3	2	3
P4	3	1
P5	4	5
P6	6	4

Round Robin Scheduling – Example4

Quantum = 2

Process No.	Arrival Time (AT)	Burst Time (BT)
P ₁	0	5
P ₂	1	7
P ₃	2	3
P ₄	3	4